

AXI4-Lite Protocol — A Study Guide (with the HDC core as the worked example)

This note explains the **AXI4-Lite** protocol from first principles, then shows exactly how `rtl/hdc_core_axi_lite.sv` implements it. The goal is that after reading it you can look at any AXI4-Lite slave, read a waveform, and understand every signal transition.

1. Why AXI exists

On a Zynq SoC the ARM cores (the **PS**, processing system) need to talk to your custom logic (the **PL**, programmable logic). They do this over a memory-mapped bus: the CPU does a normal load/store to an address, and that address is routed to your block, which looks to software like a set of registers. **AXI** (Advanced eXtensible Interface, part of ARM's AMBA family) is the standard for that bus.

There are three flavours:

Variant	Use	Complexity
AXI4-Lite	simple register access, one word at a time	low
AXI4 (full)	bursts (many words per transaction), high bandwidth	high
AXI4-Stream	continuous data flow, no addresses	medium

Our control wrapper uses **AXI4-Lite** (configure registers, kick off one inference). The future streaming path will use **AXI4-Stream + DMA**. This guide is about AXI4-Lite.

2. The one idea you must internalise: VALID/READY handshake

Every AXI channel — no matter what it carries — moves data with the **same two-signal handshake**:

- The **source** asserts `VALID` when it has put data on the wires.
- The **destination** asserts `READY` when it can accept data.
- **Transfer happens on the rising clock edge where `VALID && READY` are both high.** That edge, and only that edge, is when the data is considered moved.

Rules that follow from this:

1. Once a source asserts `VALID`, it must keep `VALID` (and the data stable) until the transfer happens. It may **not** withdraw an offer.
2. `READY` may be asserted before or after `VALID`, or simultaneously. The destination can wait (back-pressure) by holding `READY` low.
3. Neither side may wait for the other in a way that deadlocks: a source must not wait for `READY` before asserting `VALID`.



If you understand this handshake, AXI4-Lite is just **five independent channels that each use it**.

3. The five channels

A write is three channels; a read is two. They are independent — the only ordering AXI guarantees is given by the handshakes themselves.

Write transaction

Channel	Signals	Carries	Direction
AW (write address)	AWADDR , AWVALID , AWREADY	which register to write	master → slave
W (write data)	WDATA , WSTRB , WVALID , WREADY	the data + byte-enables	master → slave
B (write response)	BRESP , BVALID , BREADY	"write done / OK?"	slave → master

Flow: master presents AW and W (often together) → slave accepts both → slave returns B → master accepts B. One write = three handshakes.

Read transaction

Channel	Signals	Carries	Direction
AR (read address)	ARADDR , ARVALID , ARREADY	which register to read	master → slave
R (read data)	RDATA , RRESP , RVALID , RREADY	the data + status	slave → master

Flow: master presents AR → slave accepts → slave returns R with data → master accepts R. One read = two handshakes.

The extra signals

- **WSTRB** — one bit per byte lane (4 bits for a 32-bit bus). A **1** means "write this byte." Lets software do byte/half-word writes. Our wrapper honours **WSTRB** on the register and staging-buffer writes.
- **BRESP / RRESP** — 2-bit status: **00** = OKAY , **10** = SLVERR , **11** = DECERR . A simple slave always returns **OKAY** .
- **ACLK / ARESETN** — single clock, active-**low** reset. Everything is synchronous to **ACLK** .

4. Address decoding and the register map

AXI4-Lite is **byte-addressed**, but a 32-bit slave transfers a whole 32-bit word per transaction. So register addresses are spaced 4 bytes apart and the low 2 address bits are ignored (`ADDR_LSB = 2`). The slave compares the upper address bits against its register offsets.

Our wrapper's map (`hdc_core_axi_lite.sv`):

Offset	Name	Access	Meaning
0x000	CTRL	W	bit0 START, bit1 LOAD_PROTO, bit2 LOAD_MASK, bit3 CLR_DONE
0x004	STATUS	R	bit0 BUSY, bit1 DONE (sticky)
0x008	PROTO_IDX	RW	class index for LOAD_PROTO
0x00C	RESULT	R	(<code>class_idx<<16</code>) <code>class_dist</code> , bit31 = DONE
0x010 – 0x018	LEVELS0..2	RW	the 80-bit quantized level grid
0x100 – 0x17C	STAGING	RW	32 words = one 1024-bit hypervector

Two idioms worth noting:

- **Write-1-to-pulse control bits.** Writing `0x2` to CTRL makes the hardware emit a *one-cycle* `LOAD_PROTO` pulse, then the bit self-clears. Software never has to write a `0` back. `START/LOAD_*/CLR_DONE` all work this way.
- **A staging buffer for wide data.** A 1024-bit vector can't fit in one 32-bit write, so software writes 32 words into the STAGING window, then pulses `LOAD_PROTO/LOAD_MASK` to commit the whole buffer into the target register inside the PL. This is the standard trick for moving data wider than the bus over AXI4-Lite.

5. How the slave implements it (annotated)

`hdc_core_axi_lite.sv` runs everything in one clocked process. The key pieces:

Accepting a write. When both `AWVALID` and `WVALID` are seen, the slave raises `AWREADY` and `WREADY` together for one cycle:

```
if (!s_axi_awready && s_axi_awvalid && s_axi_wvalid) begin
    s_axi_awready  <= 1'b1;
    s_axi_wready   <= 1'b1;
    awaddr_latched <= s_axi_awaddr; // remember the address
end else begin
    s_axi_awready <= 1'b0;
    s_axi_wready  <= 1'b0;
end
```

The actual write + response. On the cycle where address and data are both accepted, it decodes `awaddr_latched`, applies `WSTRB`, and raises `BVALID`:

```
write_fire = (!s_axi_bvalid && s_axi_awready && s_axi_awvalid &&
             s_axi_wready && s_axi_wvalid);
if (write_fire) begin
    s_axi_bvalid <= 1'b1; s_axi_bresp <= 2'b00;    // OKAY
    case (awaddr_latched[11:0]) ... endcase         // update the register
end else if (s_axi_bvalid && s_axi_bready) begin
    s_axi_bvalid <= 1'b0;                          // master took the response
end
```

Reads are simpler: raise `ARREADY` to accept the address, then drive `RDATA` with `RVALID` and wait for `RREADY`.

Result capture (the bridge to the datapath). START sets `busy`, and when the core finishes the wrapper latches the answer and sets the sticky `DONE`:

```
if (busy_reg && core_out_valid) begin
    result_idx <= core_class_idx;
    result_dist <= core_class_dist;
    busy_reg    <= 1'b0;
    done_reg    <= 1'b1;    // software polls STATUS.DONE / reads RESULT
end
```

6. The software view (what the CPU actually does)

From `sw/hdc_core_axi_example.c`, a write is just `Xil_Out32(base+offset, val)` and a read is `Xil_In32(base+offset)` — the AXI handshakes are invisible to software; the interconnect performs them. One inference is:

```
hdc_wr(LEVELS0, lv0); hdc_wr(LEVELS1, lv1); hdc_wr(LEVELS2, lv2);
hdc_wr(CTRL, START);                                     // write-1-to-pulse
do { s = hdc_rd(STATUS); } while (!(s & DONE));          // poll
r = hdc_rd(RESULT);                                       // class index + distance
hdc_wr(CTRL, CLR_DONE);
```

This poll-for-done pattern is the classic AXI4-Lite control idiom (an interrupt line is the alternative to polling for lower latency).

7. How the testbench drives it (and how to read the waveform)

`tb/tb_core_axi_cosim.sv` contains a tiny AXI4-Lite **master model**: two tasks, `axi_write(addr,data)` and `axi_read(addr,data)`, that perform the handshakes exactly as above. Reading them is the fastest way to see the protocol in motion:

- `axi_write` drives `AWVALID/WVALID` and **holds them until** `AWREADY && WREADY`, then drops them and waits for `BVALID` while holding `BREADY`.
- `axi_read` drives `ARVALID` until `ARREADY`, then captures `RDATA` when `RVALID` is high.

In a waveform viewer, line up `clk` and look for the edges where a `*VALID` and its matching `*READY` are both high — those are your transfers; everything else is waiting. The co-sim runs the real PS programming sequence (load 8 prototypes + mask, then 200 inferences) and checks every predicted class index and distance against the Python golden — **200/200 PASS**.

8. Checklist / glossary

- **Transfer** = `VALID && READY` **at a rising clock edge**. Memorise this.
- **AW / W / B** = write address / write data / write response.
- **AR / R** = read address / read data.
- **WSTRB** = per-byte write enable.
- **BRESP / RRESP** = `00` OKAY (a simple slave always returns OKAY).
- **ARESETN** = active-low, synchronous reset; deassert `VALID` s in reset.
- **Back-pressure** = a destination holding `READY` low to stall the source.
- **Write-1-to-pulse** = control bit that triggers a one-cycle action and self-clears.
- **Staging buffer** = register window used to assemble data wider than the bus.
- Next step beyond this guide: **AXI4-Stream** (`TVALID/TREADY/TDATA/TLAST`) for the streaming feature path — same handshake, no addresses.